```
# ====================================
# Import libraries
# ====================================
import torch
from torch import nn, optim
import torch.nn.functional as F
from torch.utils.data import random_split, DataLoader
import torchvision.transforms.v2 as T
from torch.utils.data import TensorDataset
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import random

#NB. Multi Head Model
#Head 1 predicts the hour (12 classes: 0-11);
#Head 2 predicts the minutes (60 classes: 0-59);
#Classification for both tasks.

#NB2. I will still try to reuse as much code as possible from the previous tasks
```

```
#Check whether a GPU is available
print("GPU available:", torch.cuda.is_available())

#Check which GPU is being used
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
else:
    print("Using CPU")
```

```
GPU available: True
GPU name: Tesla T4
```

```
# ====================================
# Reproducibility and device
# ====================================
r_seed = 99
generator = torch.Generator()

def reset_seed(seed=r_seed):
    """Reset all seeds at once."""
    random.seed(seed) #reset python random seed
    np.random.seed(seed) #reset numpy random seed
    torch.manual_seed(seed) #reset pytorch random seed
    generator.manual_seed(seed) #reset the dataloader random seed
    if torch.cuda.is_available(): #if we use the gpu
        torch.cuda.manual_seed_all(seed)

reset_seed()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
#This cell runs on Google Colab if you save the data files in a folder called "data" in Drive.
#To run locally, skip this cell and set the paths to data/075_images.npy

from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# ====================================
# Load the Clock dataset (75x75)
# ====================================
images_path = '/content/drive/MyDrive/DL_Assignment_1/data/075_images.npy'
labels_path = '/content/drive/MyDrive/DL_Assignment_1/data/075_labels.npy'

images_np = np.load(images_path)
labels_np = np.load(labels_path)

#Check out our images
    #No longer needed at this point

#Convert into pytorch tensors
```

```
images = torch.from_numpy(images_np).float() #turn numpy array into pytorch tensor, float
images = images.unsqueeze(1) #add channel dimension, we need format (batch_size, channels, height, width) (greyscale = 1 channel)
mean = images.mean()
std = images.std()
images = (images - mean) / std #standardize the pixels (easier for training)

#Create two separate labels for minute and hour
labels_hour = torch.from_numpy(labels_np[:, 0]).long() #HOURS, pytorch tensor
labels_minute = torch.from_numpy(labels_np[:, 1]).long() #MINUTES, pytorch tensor
print(labels_hour.shape, labels_minute.shape) #18'000 labels for both

#We are trying to predict:
#Hours: from 0 to 11.
#Minutes: from 0 to 59.
```

```
torch.Size([18000]) torch.Size([18000])
```

```
#====================================
# Train-Validation-Test split
# ====================================
reset_seed()

dataset = TensorDataset(images, labels_hour, labels_minute) #Now we have 3 arguments
n = len(dataset)

train_size = int(0.8 * n)
valid_size = int(0.1 * n)
test_size = n - train_size - valid_size

train_data, valid_data, test_data = random_split(
    dataset,
    [train_size, valid_size, test_size],
    generator=generator
)

batch_size = 64

train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    shuffle=True,
    generator=generator
)

valid_dataloader = DataLoader(
    valid_data,
    batch_size=batch_size,
    shuffle=True
)

test_dataloader = DataLoader(
    test_data,
    batch_size=batch_size,
    shuffle=True
)
```

```
# ====================================
# Setting up the CNN
# ====================================

class Net_4conv(nn.Module): #Final CNN
    def __init__(self): #we try to use: 12 classes for hours, 60 classes for minutes
        super().__init__()
        #Input image (1, 75, 75) (greyscale, 75x75)

        #Same layers for both
        self.conv1 = nn.Conv2d(1, 6, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 3, padding=1)
        self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
        self.conv4 = nn.Conv2d(32, 64, 3, padding=1)
        self.fc1 = nn.Linear(64 * 4 * 4, 120)
        #Two separate outputs
        self.hour_head = nn.Linear(120, 12) #head 1: hour
        self.minute_head = nn.Linear(120, 60) #head 2: minute

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
```

```
        x = self.pool(F.relu(self.conv4(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))

        #two separate outputs
        return self.hour_head(x), self.minute_head(x)
```

```
# ======================================================
# Defining the functions for training and evaluation
# ======================================================

#Train the model

def train_model(model, train_dataloader, optimizer, criterion, epochs, alpha): #alpha for combining the losses

    """Train a model for a given number of epochs."""

    for epoch in range(epochs):

        model.train()
        running_loss = 0.0

        for inputs, hours, minutes in train_dataloader:

            #data agumentation
            #inputs = train_transform(inputs)

            #To use external GPU
            inputs = inputs.to(device)
            minutes = minutes.to(device)
            hours = hours.to(device)

            #zero the parameter gradients
            optimizer.zero_grad()

            #forward + backward + optimize
            out_hour, out_minute = model(inputs) #predictions
            loss_hour = criterion(out_hour, hours) #Loss for HOURS
            loss_minute = criterion(out_minute, minutes) #Loss for MINUTES
            loss = loss_hour + alpha * loss_minute #COMBINE the Losses
            loss.backward() #compute gradients
            optimizer.step() #optimize weights

            #compute running loss
            running_loss += loss.item()
        print(
            f"Epoch {epoch + 1}/{epochs}: "
            f"- Loss = {running_loss/len(train_dataloader):.4f}"
        )

#Validate the model

def evaluate_model(model, dataloader):

    """Evaluate a model using the average circular time error."""

    model.eval()

    total_error = 0
    total = 0

    with torch.no_grad():
        for inputs, hours, minutes in dataloader:

            #To use external GPU
            inputs = inputs.to(device)
            minutes = minutes.to(device)
            hours = hours.to(device)

            #Get predictions and classes
            out_hour, out_minute = model(inputs) #predictions
            predicted_hours = torch.argmax(out_hour, dim=1) #Class for HOURS: assign to the class with the highest score
            predicted_minutes = torch.argmax(out_minute, dim=1) #Class for MINUTES: assign to the class with the highest score

            #Combine predictions and true times into total minutes
            predicted_time = predicted_hours * 60 + predicted_minutes #total number of minutes
            true_time = hours * 60 + minutes #total number of minutes

            #Calculate their absolute difference and their circular difference
            difference = torch.abs(predicted_time - true_time)
            circular_difference = torch.minimum(difference, 720 - difference)
```

```
            #Compute the errors
            total_error += circular_difference.sum().item()
            total += minutes.size(0)

    common_sense_error = total_error / total

    return common_sense_error

#NB.
#While we use two separate Losses to train the model (and then combine them into one),
#We use one common sense error to check how off our predictions are.
#Therefore for the common sense error we compute again the total number of minutes, and
#the circular difference, because it's convenient to estimate the error in our predictions.
```

```
# ====================================
# Experiment with different alpha values
# ====================================

learning_rate = 0.001
epochs = 20

#Alpha values: the value with which we combine the
#two losses: L_tot = L_hour + alpha*L_minute
alpha_values = [0.5, 1.0, 2.0]

results, models = [], {}

for alpha in alpha_values:

    print(f"\nTraining with alpha = {alpha}")

    #New seed for every experiment
    reset_seed()

    #New model for each experiment
    model = Net_4conv().to(device)

    #Loss function
    criterion = nn.CrossEntropyLoss()

    #Optimizer
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    #Train the model
    train_model(
        model,
        train_dataloader,
        optimizer,
        criterion,
        epochs,
        alpha
    )

    #Validate the model
    valid_error = evaluate_model(
        model,
        valid_dataloader
    )

    models[alpha] = model

    #Save results
    results.append({
        "Alpha": alpha,
        "Validation common-sense error (min)": valid_error
    })
    pd.DataFrame(results).to_csv("results_multihead.csv", index=False)

results_df = pd.DataFrame(results).sort_values(by="Validation common-sense error (min)")

print(results_df)
```

```
Epoch 10/20:  Loss = 1.2445
```

```
Epoch 19/20: - Loss = 1.3445
Epoch 20/20: - Loss = 1.3052

Training with alpha = 1.0
Epoch 1/20: - Loss = 6.5815
Epoch 2/20: - Loss = 6.5803
Epoch 3/20: - Loss = 6.5801
Epoch 4/20: - Loss = 6.5800
Epoch 5/20: - Loss = 6.5799
Epoch 6/20: - Loss = 6.5798
Epoch 7/20: - Loss = 6.5797
Epoch 8/20: - Loss = 6.5797
Epoch 9/20: - Loss = 6.5797
Epoch 10/20: - Loss = 6.5796
Epoch 11/20: - Loss = 6.5795
Epoch 12/20: - Loss = 6.5795
Epoch 13/20: - Loss = 6.5795
Epoch 14/20: - Loss = 6.5795
Epoch 15/20: - Loss = 6.5795
Epoch 16/20: - Loss = 6.5795
Epoch 17/20: - Loss = 6.5795
Epoch 18/20: - Loss = 6.5795
Epoch 19/20: - Loss = 6.5795
Epoch 20/20: - Loss = 6.5794

Training with alpha = 2.0
Epoch 1/20: - Loss = 10.6772
Epoch 2/20: - Loss = 10.6755
Epoch 3/20: - Loss = 10.6750
Epoch 4/20: - Loss = 10.6749
Epoch 5/20: - Loss = 10.6747
Epoch 6/20: - Loss = 10.6680
Epoch 7/20: - Loss = 10.1898
Epoch 8/20: - Loss = 8.9927
Epoch 9/20: - Loss = 8.1070
Epoch 10/20: - Loss = 7.2873
Epoch 11/20: - Loss = 6.6326
Epoch 12/20: - Loss = 6.1433
Epoch 13/20: - Loss = 5.7783
Epoch 14/20: - Loss = 5.4903
Epoch 15/20: - Loss = 5.2398
Epoch 16/20: - Loss = 5.0411
Epoch 17/20: - Loss = 4.8845
Epoch 18/20: - Loss = 4.7130
Epoch 19/20: - Loss = 4.5773
Epoch 20/20: - Loss = 4.4826
   Alpha  Validation common-sense error (min)
0    0.5                            13.855000
2    2.0                            29.930556
1    1.0                           181.480000
```

```
# ====================================
# Test set - With Best Parameters
# ====================================

#Best alpha
best_alpha = float(results_df.iloc[0]["Alpha"])

#Evaluate the final model on test set
test_error = evaluate_model(
    models[best_alpha],
    test_dataloader
)

print(f"Best alpha: {best_alpha}, test error: {test_error:.2f} minutes")
```

```
Best alpha: 0.5, test error: 12.96 minutes
```

```
# ====================================
# Save Results
# ====================================

final = pd.DataFrame([{
    "Formulation": "Multi-head",
    "Setting": f"alpha = {best_alpha}",
    "Epochs": epochs,
    "Validation error (min)": results_df.iloc[0]["Validation common-sense error (min)"],
    "Test error (min)": test_error
}])

print(final)
```

```
  Formulation      Setting  Epochs  Validation error (min)  Test error (min)
0  Multi-head  alpha = 0.5      20                  13.855         12.963333
```

```
# ====================================
```

```
# Compare performances
# ====================================

#Compare the performance of:
# - Multi Head (separate min and hour)
# - Classification (together min and hour)
# - Regression (together min and hour)

final = pd.DataFrame([{
    "Classification": "best_bin_size = ??, epochs = 20, Validation common-sense error (min) = ???, Test error (min)",
    "Regression": "best_loss = SmoothL1Loss, epochs = 20, Validation common-sense error (min) = ???, Test error (min)",
    "Multi-head": "best_alpha = 0.5, epochs = 20, Validation common-sense error (min) = ???, Test error (min)",

}])

print(final)
```

```
                                    Classification  \
0  best_bin_size = ??, epochs = 20, Validation co...

                                        Regression  \
0  best_loss = SmoothL1Loss, epochs = 20, Validat...

                                        Multi-head
0  best_alpha = 0.5, epochs = 20, Validation comm...
```